

Skewed Distribution Algorithm for Practiced Task Performance Forecasting

Written by:

Ian Padovani

For:

FRC Team 4206

Last Updated:

11/16/2025

Version:

2.1.2025



Table Of Contents

Background.....	2
Foundational Principles of a Hybrid System.....	3
Historically Used Algorithms.....	3
Skewed Distribution.....	6
Basic Setup.....	6
Skewed Distribution Derivation.....	7
Another Derivation Method.....	12
Simulation.....	14
Limitations.....	15
Conclusions and Recommended Improvements.....	16

Background

In many activities, there is a need for a way to quantify the performance of individuals and their ability to do a task. This can be seen in modes as casual as sports and as complex as worker allocation for a factory. Being able to quantify this kind of data allows a user to more efficiently organize and rank the performance of given individuals to make data-informed decisions. While trivial in concept, it is the focus of decades of research due to the complex nature surrounding how things happen. There are three types of systems you can conform a data model to when talking about forecasting: open systems, closed systems, and hybrid systems.

Open systems are the most versatile, yet also the most difficult to model. An open system is one where the performance of the system is affected by some random external variable(s) but the scope is comparably massive. These variables are system-specific and usually described as random due to the lack of predictability. For example, the weather is an open system. We can predict a rough path for how clouds will move or form but due to wind, heat differentials, and terrain variability, there is no guarantee a set of clouds moving over an area will act the same every time. In order to yield a better model, the scope of the system must be constrained down. This is where closed systems come in.

Closed systems are where almost all random external variables are eliminated at the cost of forecast scope such that the end result yields a much more accurate result. Limiting scope in some scenarios is sub-optimal but in other cases, it works perfectly. These systems are incredibly limited in use but are very easy to model at a high level. Take a robot arm for example. In order to get the arm to move to an exact position as fast as possible we need to predict the performance of the motors, gearboxes, and the load on the arm. All of these are controlled variables meaning there is little to no external randomness being applied. This means this robot arm is a closed system and thus predictable to a high degree. Where these systems fall apart is that they have a very limited scope and they can never be applied to humans. This is where hybrid systems are used.

Hybrid systems are the most commonly used and while easier than open systems are still complex. These systems involve taking a closed system and allowing specific external randomness to be input. As a result, the system is incredibly sensitive but allows the introduction of a human element to be added and accounted for. This allows the usefulness of the model to be orders of magnitude greater than that of a closed system with models being able to be used for completely unrelated fields with very little modification to foundational concepts. As previously mentioned, however, the system is sensitive and many people introduce a lot of inefficient methods that break some foundational requirements that need to be followed to yield an accurate model.

Foundational Principles of a Hybrid System

The first principle of a hybrid system is that an absolute result is impossible. Due to the randomness input it is unreasonable to expect an absolute result. For example, if a human is operating a machine, they could be having an “off-day.” Their performance will not be the same day to day so as such, declaring an absolute result is unrealistic and useless. Instead, a result that declares some aspect of variance shows an expected range of performance that is realistic for the average day. While setting an extreme variance guarantees higher accuracy, it also discredits the algorithm. Typically, you will see systems declare one standard deviation or two standard deviations away from the average in order to cover the variance.

The second foundational principle of a hybrid system is that forecasted values should be made relativistic. Non-relativistic values suffer from non-proportional inflation when being forecasted because values on the smaller end no longer maintain their relationship to larger values. Additionally, it becomes impossible to optimize as you can only optimize for a section of data instead of the entire data set. This is inefficient in that it becomes accurate exclusively to the section of data it was optimized for at the cost of everything else becoming worse. In the past, it was reasoned that this was a natural loss inherent to analysis but, as will be proven later, is wildly inaccurate. By making forecasted values relative, you maintain existing relationships and more uniformly deal with the data. Utilizing the aforementioned benefits of a closed system, it is best to choose your maximum value for the relativistic base to be as close to a unified closed system as possible. The example studied in this paper shows an example of this where instead of being relative to every competition being run it is only run relative to a specific competition and division. This closes the system to limit outside factors that become irrelevant for the use case of what is being forecasted at the cost of the data being unusable between different competitions and even different divisions at the same competition.

The third and final foundational principle of a hybrid system is that 100% accuracy is impossible for an optimized algorithm. If an algorithm achieves 100% accuracy then this means the variance band is inflated and can be narrowed. This principle establishes the fact that there is no such thing as a single optimal solution. So when optimizing, it is important to know when to stop in terms of usefulness.

Historically Used Algorithms

Several different algorithms have been utilized in the past for various rankings in performance. One of the most popular methods is a system called ELO. This system is used very widely in chess as a way to rate players. The basic idea is that each player starts with a rating of a certain number of points. As players go against each other, they gain or lose points based not

only on if they win or lose but how difficult of an opponent they go against. While fairly well implemented, it still has a few issues. Sample size, inflation, selective pairing, and generalization. A similar system called MMR is commonly used in video games and other outlets with randomized matchmaking. In principle it works the exact same as ELO where each player starts at a certain point value and then increases it with a win, or decreases it with a loss where the amount varies with opponent difficulty. Since it is so similar it suffers the same issues as ELO. OPR is another system that can be seen used commonly in sports and even in the white-collar workplace for worker performance. The system works via one massive system of equations where each individual player is a variable and variables are added together to get a bunch of different totals. This system also suffers from sample size, inflation, selective pairing, and generalization. A more detailed explanation of these different systems is not within the scope of this white-paper as there is extensive material available that describes how they work. The common issues will be discussed however as they are relevant to why they perform sub-optimally in select cases.

The first issue, sample size, is a common issue among any algorithm. There is some baseline level amount of data required to be able to function at a minimum level of accuracy. The more data, the better it gets, however a common mode of usability for an algorithm is determined by how quickly it can achieve its asymptotic accuracy level. Since no algorithm should ever reach 100% accuracy, there should be some accuracy that it slowly approaches as you get more and more data. For example, if an algorithm yields an accuracy of 90% with 50 points of data and has an accuracy of 90.1% with 5000 points of data then it can be seen that it has reached its asymptotic accuracy. When it comes to sports related rating systems, many suffer from needing every player to face each other to get a high level of accuracy. For many sports, this is unrealistic and for some it is impossible. As a result, the accuracy suffers. To combat this, many sports implement a range based ranking method where they group players by rating size with the logic being that every player roughly in this band of rating is confidently better than a lower band. This too has the issue of how to handle the extremes of each band. A player at the bottom of a band is fairly close to a player at the top of a lower band. The end result is a gray area between bands which can be inaccurate.

The second and third issues go hand in hand. Inflation and selective pairing, even with randomized matchups, can exist and cause rankings to not be true. This occurs because of the aforementioned gray areas on the ranking bands. Since these areas are so heavily contested and narrow, a lot of movement occurs which can inflate the “amount of rating” in play. In chess for example, this occurred towards the bottom of the rating system where new players kept unintentionally boosting players as more ELO points kept being introduced. As a result, the number of chess players above a certain ELO has increased making the ranking not as representative. While this could be a result of popularity increasing in recent years, the rating bands are not increasing equally. No studies have been conducted on this phenomenon but the

inflation is generally recognized by the community involved. Selective pairing is an issue that also goes hand in hand with inflation. Players will choose who they go against so they can boost their rating. Even in randomized matchmaking this still occurs due to a varied level of effort and many other complex factors like luck. That being said, randomized matchmaking heavily reduces the impact of selective pairing usually.

The fourth and final issue is generalization. Remember that the second principle of a hybrid system is that it is relativistic. ELO, MMR, and OPR are all generalized algorithms due to the rating systems having to inherently work for a broad span of players in different sports. No player's performance is made relative to any other so when two unrelated players face each other the rating system becomes fairly inaccurate. Many of these systems are only able to see accuracy ratings around 78-80% when well implemented and 82-83% in some very niche cases. By focusing on a more relativistic method, a considerably higher accuracy can be achieved at the cost of being more difficult to generalize.

Skewed Distribution

For a long while, it has been assumed that humans perform within a normal distribution. A normal distribution, also known as a bell curve, is a curve that is created by a perfectly random data set that is based around some average and standard deviation. It is considered to be a gold standard but suffers from having to be perfectly random. In almost all cases, nothing is perfectly random when it comes to human performance in sports. The players practice outside of competing and will bias to some level of performance more than a perfectly random level of performance based around some average. This causes a skew in the curve meaning the curve is no longer symmetric. Being no longer symmetric, standard probability calculations can not be implemented to predict percentages. Previous implementations of this algorithm ignored this skew and forced a normal distribution. This was only capable of achieving 85% on average and was deemed not good enough. Similar to ELO, a log-normal distribution was considered as this better fits the data to the skew but was also found to be only able to achieve around 86% accuracy. This performance was largely unacceptable to the previously mentioned requirements so the conclusion was that skew needed to be accounted for and used. In a perfect world, utilizing this method should yield a highly accurate end result.

Basic Setup

Setting the baseline, a player's performance will be categorized into performance aspects that are to be tracked. These aspects need to be an objective metric of some sort that directly correlates to the matchup outcome. These can be as simple as points scored or as complex as derived statistics. Each of these statistics will be given their own skewed distribution. Any interconnecting statistics need to maintain their connection to ensure a continued level of realism when simulated. For example, if a player has to score 3 objects in a basket in order to even have the chance to score an object in a different basket, this connection needs to be maintained. This will be explored further in detail later on when the method of simulation is discussed.

Each statistic needs to be tracked with a high level of accuracy or the data will become useless very quickly. So picking statistics that are easy to track and limiting scope to a level that will be consistent is the key. Many modern sports utilize a digital based system to track everything which has proven to be incredibly accurate and beneficial. However, smaller events and competitions do not have this luxury and usually rely on specific people watching the players and marking down how they do. In this case, scope becomes incredibly important. Errors will still happen regardless of the scouting method so many programs that use human-input for data will use an error checking function to ensure data quality. These functions are typically based on general averaging functions or a heavily simplified OPR algorithm where if a value is determined to be too extreme it is flagged for review. While useful, this approach is best avoided

to limit the complexity of what is being done. Even if data points are off by a bit, they can usually be dampened out of the system via the simulation and pre-filtering outliers.

Once the data is sorted and acquired, a pre-filter can be applied. However, for most cases this pre-filter will need to be experimentally optimized. The tested use case described later was able to have a filter excluding the lowest and highest value and in some cases only excluding the lowest value. Other cases may need more extensive filtering done depending on the nature of the data itself. After being filtered, an average, standard deviation, and skew must be calculated for each statistic being tracked. This will be used to determine the curve for a given statistic in the simulation later on. For calculating the necessary values: the equation for calculating the average can be seen in equation (1), standard deviation can be seen in equation (2), and skew can be seen in equation (3).

$$\mu = \frac{\sum_{i=1}^n x_i}{n} \quad (1)$$

$$\sigma = \sqrt{\frac{\sum (x_i - \mu)^2}{n}} \quad (2)$$

$$\gamma = \frac{n}{(n-1)(n-2)} \sum_{i=1}^n \left(\frac{x_i - \mu}{\sigma} \right)^3 \quad (3)$$

Where μ is the mean, σ is the standard deviation, and γ is the skew. It is important to note that many modern software packages have these basic statistical functions built in and should not require coding the equation by hand.

Skewed Distribution Derivation

Now that setup is done, a curve will be generated to characterize the performance of a player. The end goal being a skewed distribution curve that roughly models the expected performance of a player. This method is not perfectly accurate so it will be dampened using a forced normal distribution. While this dampening will result in an under-estimate for higher performing players and overestimation for lower performing players, the overall system should be more robust. This robustness will yield an algorithm that will not make as extreme of decisions in deciding the winner of a matchup which consequently also will be better at predicting truly close matchups.

The first step is understanding a normal distribution and why it is being used. As previously mentioned, a normal distribution is a natural curve generated by the combination of several perfectly random variables. It shows up almost everywhere and has been extensively used in a wide variety of setups due to it being an occurrence of nature. An example can be seen below of what one of these looks like.

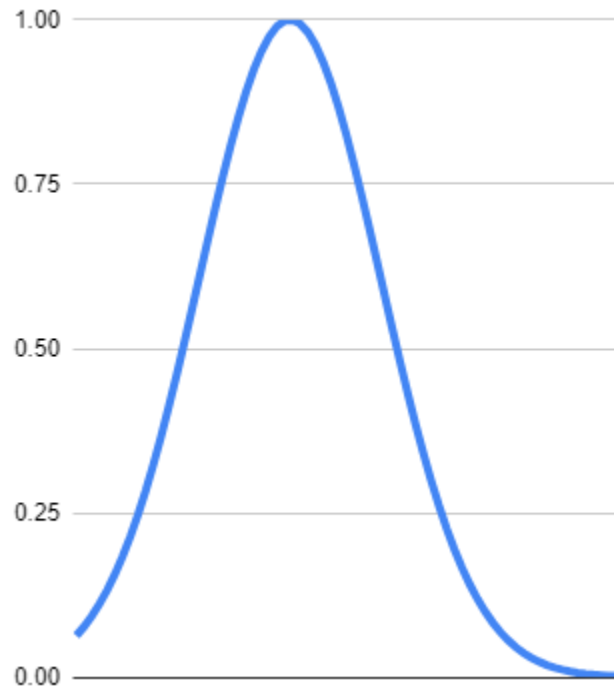


Figure 1. Normal Distribution

For simplicity of comparison, the graphs will be made relative to their maximum value. Normally on these graphs, the x-axis represents the incremental performance being observed and the y-axis is the weight of the respective x-axis. In other words, if an x-value of 15 is more likely than a value of 12, 15 will have a larger y-axis value. It is important that the y-axis is not the probability. The actual percentage probability is calculated via integrating the curve such that the bounds are the two x-values being observed. The equation for a normal distribution can be seen in equation (4) and the probability can be seen in equation (5).

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2} \quad (4)$$

$$P(x_{i1} \leq x \leq x_{i2}) = \frac{\int_{x_{i1}}^{x_{i2}} f(x) dx}{\int_{-\infty}^{\infty} f(x) dx} \quad (5)$$

These equations have been vastly simplified with modern computing, such that most programs have an easy way to calculate both of these without inputting the equations. In many spreadsheet programs a function called NORMDIST where given the mean, standard deviation, and an x-value will output the correct y-value to graph a normal distribution. To calculate the probability many programs include a function similar to ZTEST that calculates the probability from infinity to your specified x-value. It is important to mention that these same programs include a function similar to NORMINV which calculates the inverse normal distribution of a function. By providing a number between 0 and 1, the mean, and the standard deviation the function will calculate the x-value that corresponds to the number you specified. So if 0.5 is given as the value to find, it will find the 50th percentile x-value in a normal distribution modeled by the specified mean and standard deviation.

As can be seen, a normal distribution is fairly easy to find with modern tools and a given data set so long as the average and standard deviation are found. Force fitting a normal distribution to a data set can be done this way but will yield mixed results. Recalling the theory behind a normal distribution, it relies on perfectly random data. Nothing in the world is perfectly random as there will always be some bias or causation. That being said, some things happen to get close enough that it is good enough. To determine how “close” a data set is to being normal the skew is utilized and can be seen in equation (3). For a data set to be close to normal the skew value should be 0. A positive value of skew means the data is left skewed or lower x-value skewed, while a negative value of skew means the data is right skewed or higher x-value skewed. If the data is skewed it can not be assumed to be normal and thus the above method of generating a model for a normal distribution will not work or yield accurate results. So in order to account for it we must do a skew-adjustment per this paper. Figure 2 shows the difference between a normal distribution (blue) and a fully skew adjusted distribution (red).

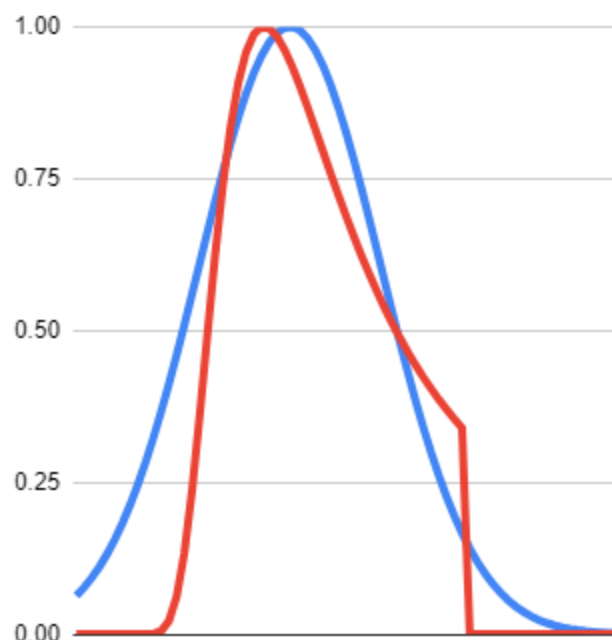


Figure 2. Comparison of a normal distribution and a skew adjusted distribution

As can be seen in figure 2, the skewed distribution provides a much more distinct curve that will better account for human performance. A player that is constantly practicing a set task will bias to some minimum level of performance and is more likely to perform better than to perform horribly. This also shows that the original data set had a positive skew since the peak of the curve is left biased. Another observation is the difference between the normal distribution and the fully adjusted skewed distribution. The normal distribution over-valued the lower end of x-values and the middle section of x-values, meanwhile it under-valued the upper end of x-values. Finally, notice the sharp cutoff in the upper x-value range. This is determined via several factors like compatibility, improvement, and more to be discussed in detail later. The reason it is used is that there is logically some maximum that exists for a player. While they could perform above the maximum, there is no logical justification for it based on the data set. This new cutoff is determined to be the maximum reasonable value for a given data set and usually falls within the 90-98th percentile value of the original normal distribution but can differ from this with edge cases. Proven accuracy difference is an improvement from ~84-86% with the normal distribution to ~90-96% with the fully skew adjusted distribution in the tested environment of high school robotics.

To do the skew adjustment, there are three options that must all be calculated where D is the resultant array of x-values after adjustment. The skew of the three that is closest to zero will be the method utilized for the curve. The first method is the square root method. This method takes the existing data set and finds the square root of all of the values according to the equation

(6). Notice the difference for if the skew is positive or negative. This is important in getting accurate adjustments.

$$D_{sq}[x_i] = \left\{ \sqrt{x_i} ; \gamma \geq 0 \right\} ; D_{sq}[x_i] = \left\{ x_i^2 ; \gamma \leq 0 \right\} \quad (6)$$

The second method is the Log base 10 method. This is almost identical to the previous method but replacing the square root with a base 10 logarithm.

$$D_{lg}[x_i] = \left\{ \log(x_i) ; \gamma \geq 0 \right\} ; D_{lg}[x_i] = \left\{ x_i^3 ; \gamma \leq 0 \right\} \quad (7)$$

The final method is the exponential decay method. Once again, this is almost identical but changes the logarithm with a reciprocal.

$$D_{ed}[x_i] = \left\{ x_i^{-1} ; \gamma \geq 0 \right\} ; D_{ed}[x_i] = \left\{ x_i^4 ; \gamma \leq 0 \right\} \quad (8)$$

$$D_{final}[x_i] = \min_D \left(\left| 0 - \gamma_{\{D_{sq}, D_{lg}, D_{ed}, D_{base}\}} \right| \right) \quad (9)$$

Where D_{final} is the D array of x-values, including the original unadjusted data set, with a skew closest to zero. Using this adjusted data set, an average and standard deviation can be determined. It is important to understand that these adjustments affect your entire scale. Meaning that if your original x-axis was every value from 1 through 25 increments of 1 but you did an exponential decay adjustment for a positive skew your new x-axis is the original scale and original order where each value is its reciprocal. So 1 is still 1 but 2 becomes 0.5 and 4 becomes 0.25 while still maintaining the same order (i.e. {1,2,3,4} becomes {1, 0.5, 0.3333, 0.25}). This is crucial to understand or the distribution will not be graphed correctly at all. Coincidentally, this also means that when calculating the probability of a certain value range, the skew adjustment method must be known and utilized accordingly. A minimum cutoff needs to be applied at zero as a negative performance is not possible. For the sake of progression, assume a maximum cutoff at the 95th percentile of the normal distribution.

The final step is to generate a graph that shows the curve so that it can be directly compared to the normal distribution. To do this, take the generated y-values from the normal distribution and

divide each by the maximum y-value and set the x-axis to equal to the same normal distribution x-axis. This will mean that the peak value will be 1 and all other values will be some decimal relative to 1 that maintains the shape of the normal distribution into a new graph that for the sake of simplicity will be called a scaled normal distribution. Next, take the y-values of your skew adjusted curve and divide them by the maximum y-value of the skew adjusted data set in the same way that was done with the normal distribution. These new y-values can be graphed using the same x-axis as the scaled normal distribution. The final curve can be called the scaled skew adjusted distribution. This results in the ability to graph both on the same graph and compare them. The advantage to going all the way through to the scaled skew adjusted distribution is that now the number of inverse calculations is reduced and your probability is directly related to the original desired x-values. As previously mentioned, Figure 2 is the direct outcome of the process. Two aspects not accounted for when scaled are the mean and standard deviation. The mean will still need to be calculated via the inverse of the adjustment method but the standard deviation will need to be approximated as shown in equation (10).

$$\sigma_{adj} \approx \sqrt{|\left(\mu_{base}\right)^2 - \left(\mu_{adj}\right)^2|} \quad (10)$$

Where the base mean is the mean of the original data set and the adjusted mean is the mean of the adjusted dataset. As stated, this is an approximation of the standard deviation and works for all adjustment methods. While not a perfect metric it is close enough in practice to be utilized.

The reason this approximation method works so well is that it is taking four completely different adjustment approaches and is both relativistic and adaptive. The final step that modifies the scales such that different curves could be graphed on the same graph keeps everything such that two completely different methods could be compared accurately. Since each forecast calculation re-calculates the adjustment, the algorithm will auto adapt to changes in data trends to the method that best fits what is being observed. The end result is a very accurate and adaptive way to relativistically compare multiple skewed distributions in a novel way. Leading back to characterizing human performance, this becomes a very accurate comparison tool in forecasting performance capable of high accuracy with minimal data population requirements.

Another Derivation Method

The aforementioned method is an approximation created through testing. A more exact method exists but relies on a significantly larger data set and a much more complicated calculation. The approximation has proven to work with as little as 4 data points per player but approaches its asymptotic performance at around 10 data points per player due to several assumptions being made on how humans perform practiced tasks. More exact methods typically

require 100 data points or more since they are not making the same assumptions. While this exact method was not used in testing, it is important to at least mention it for future applications.

There are three main parameters used in this calculation and the approximation of these factors is incredibly complex. Typically the factors are not calculated and are instead curve-fit to an existing data set hence why the method requires so many more data points. The first of these parameters is called the shape parameter (α). This parameter controls the symmetry of the distribution such that a negative value represents a left-ward skew of the data and a positive value represents a right-ward skew of the data. This parameter's effect is best seen in the cumulative distribution plot seen in figure 3.

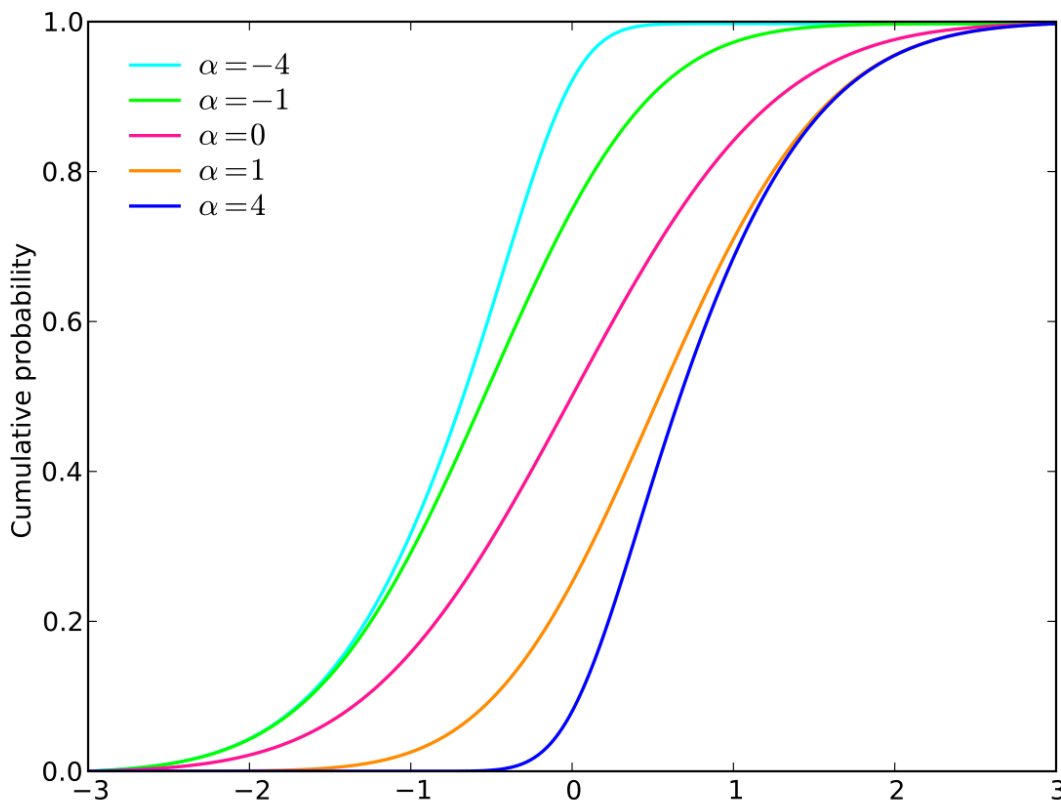


Figure 3. Cumulative Distribution Plot with a change in α

The second parameter is called the size parameter (ω) and as it is named, affects the spread of the distribution. A value of 1 indicates that a distribution that is the same size and a value of 2 indicates double the size. By decreasing or increasing the spread, the shape can be better controlled for a given skew to be more accurate. The final parameter, is the location parameter (ξ) which affects the apex location of the distribution curve. This value is directly related to the size parameter in order to maintain the correct mode of the data. Typically, the shape and size are defined then the location is added in to correct the curve fit until the mode is

correct. In order to graph these data points, equation (11) is used but it has been broken up into pieces for readability.

$$f(x) = \frac{e^Z(1+erf(\varphi))}{\omega\sigma\sqrt{2\pi}} \quad (11)$$

$$Z = \frac{-\left(\left(\frac{x-\xi}{\omega}\right)-\mu\right)^2}{2\sigma^2} \quad (12)$$

$$\varphi = \frac{\alpha\left(\frac{x-\xi}{\omega}\right)}{\sqrt{2}} \quad (13)$$

It is important to notice that utilizing these parameters means an iterative process to properly implement and the parameters will need to be reiterated per player analyzed. There does not currently exist a novel way to calculate them unless very specific requirements are met which becomes difficult to apply. This iteration results in a solving method that is not feasible at scale unless computing power is not an issue. The error function being utilized is another source of approximation though with modern software there are usually built in functions for calculating it to a high degree of accuracy. Overall, this method is largely unnecessary and over complicated in implementation at a high level which is why the approximation was introduced.

Simulation

In order to simulate the data set, a monte carlo style simulation is the usual approach. This simulation method has limitations in that it is computationally intensive. Since we already know our probability curves a more computationally efficient method can be used. Some background as to why will help provide clarity though. A monte carlo style simulation generates several random x-values and after a certain number of iterations comes out with a most probable result. This means that the resultant value will converge to a most probable x-value, also called the mode. In many cases where we do not know our probability curve, we have to use a monte carlo or similar to figure out the mode. With this algorithm, we are generating our curve which means we can find the mode very easily by observing where our scaled skewed distribution reaches a y-value of 1. This value of 1 is the peak of the curve and tells us the most probable x-value fully bypassing the need for a monte carlo. Only utilizing the skewed distribution in the simulation is not ideal though. It is better to dampen the result with a normal distribution to prevent over and under estimation.

If we go back to figure 2 where we show the two probability curves overlapped, it can be seen that there exists an intersection point between the peaks of both graph types. While both methods have similar peak values, this intersection on where the normal distribution and the skewed distribution meet in between their peaks is crucial. It represents the most probable x-value between both methods of probability. By using this value as our mode, we can achieve an optimally damped system every time. The bounds of our variance can then be determined in a similar fashion. Where the two curves intersect to the left and right of the peaks defines the most probable variance values. All together, by graphing our scaled normal distribution and our scaled skewed distribution, we can find the three locations the two graphs intersect to find our most probable value and our variance limits. There are scenarios where the variance bound intersections do not exist. In this case, it is best to use the standard deviation of our normal distribution. The reason for this is that the value is closest to the real life standard deviation that has been measured so far.

Limitations

While this algorithm has proven useful, there are limitations to its ability. The most prominent limitation is part of why it is able to work: For a limited data population size it requires a lot of high level assumptions. Larger data sets don't require these same assumptions but these larger data sets don't need approximations like this as there are more accurate methods. This feeds into the next limitation, this algorithm is only meant for a heavily limited population size. As the population size increases there exists a point where already existing curve-fit algorithms will work significantly better. This crossover point seems to be around a population size of 100 but may require more if the data does not correlate strongly enough. Another limitation is that this algorithm in its current state relies on individual performance. While individual performance is incredibly useful, teams will utilize multi-player strategies that in the end hurt the accuracy of the algorithm's predictions. The more involved the strategy the worse the accuracy becomes. For example, if a set of players are really good when paired together but perform horribly when separated, the algorithm will struggle to accurately model their performance since it is not individualized. Building off of this limitation, this means that the observational area of the algorithm has to be scoped correctly in order to work. The algorithm can not make accurate predictions for players that are unrelated. This is because, while they compete in the same sport, they are in completely unrelated divisions. As a result there are no reasonable connections for the algorithm to predict off of. Finally, the last limitation is that narrowing the point range for a score prediction becomes exponentially more difficult as it decreases. This means that there is a limit to how narrow of a prediction it can make which can be problematic if the score is not high enough relative to the granularity.

Conclusions and Recommended Improvements

Currently, in the realm of FRC robotics, the algorithm is able to achieve mid to low 90% accuracy in low strategy games and mid to high 80% accuracy in high strategy games. It will consistently and reliably achieve about 10% higher accuracy than the best existing methods currently used in FRC regardless of game. Future iterations should only improve it further albeit in smaller steps via marginal increases in accuracy and narrowing of the predicted score range. These small improvements however are instrumental in an algorithm that will hopefully define a new standard in practiced task performance forecasting.

Some steps forward to advance the algorithm are a full implementation in python so that it is not limited by spreadsheet software. This will also assist in ease of use as it will no longer require an internet connection to use other than moving raw data around. Implementing the code in python, if set up correctly, also means that the algorithm can be more rapidly applied to other areas more efficiently.

Some notes on implementing a way to account for strategy that haven't been tested but may be leads for future developments. From an observational level, strategies are a reactionary practiced task but have too many micro-interactions to efficiently turn into a metric. In theory, a better approach is to utilize the compatibility that was calculated for the FRC teams to determine a defensive power and offensive power for each player. Using these power metrics, they should directly relate to the scoring ability and thus an interaction result can be determined. By iterating through all possible combinations of interactions between players using these powers, a prediction can be made on what a likely approach will be strategy wise in terms of scoring ability. This "likely approach" should exist as a number that can be multiplied by the score to increase or decrease it accordingly. For example, a team if left undefended will likely just score at their maximum cutoff. However, as they get defended, that score "multiplier" increases or decreases which in turn changes the score. In theory, by using the previously determined "likely approach" and multiplying it by the average score, a very accurate score prediction can be made. The algorithm will have to change to accommodate this as the curve will be further modified by this new metric. It will be a fairly extensive change but may be the key to getting an algorithm that is not weak to heavy multiplayer strategy.